

Evolutionary Algorithms: Final report

Ana Francesca Stama (r0974957)

1 Metadata

- **Group members during group phase:** Ana Francesca Stama and Arent Steeno
- **Time spent on group phase:** 20 hours
- **Time spent on final code:** 50 hours
- **Time spent on final report:** 15 hours

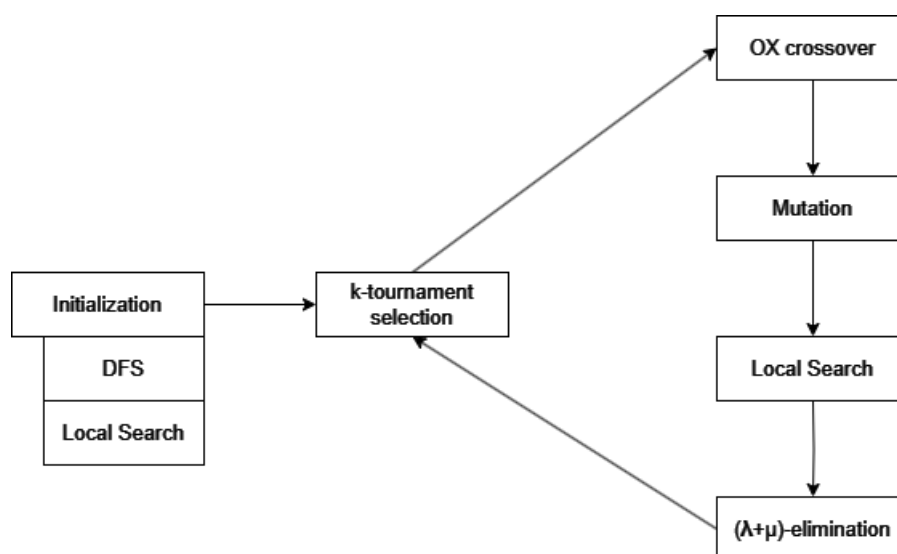
2 Changes since the group phase

1. Added an additional constraint to representation
2. Introduced an algorithm for finding a legitimate path in initialization
3. Added alternating mutation operators
4. Changed certain hyperparameters, such as the size of population, probability of mutation, k in k -tournament selection, etc.
5. Introduced diversity promotion
6. Introduced local search
7. Introduced Numba

3 Final design of the evolutionary algorithm

1. Diversity promotion
2. Mutation operators
3. Crossover operator

3.1 The main loop



3.2 Representation

The candidate solutions are represented as an order of nodes, always starting with the node 0. E.g. the order $[0, 3, 4, 1, 2]$ represents the cycle $0 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow 0$. The main advantage of this representation is its simplicity, not just in term of representation itself, but also in term of further computations (such as variation

operators) that rely on it. Initially, during the group phase, there was no constraint that the starting node has to be 0, which created the possibility that different candidate solutions could essentially represent the same cycle. E.g. candidate solutions $[0, 1, 2, 3, 4]$ and $[1, 2, 3, 4, 0]$ represent the same candidate solutions, even though they have different representations. This specific representation was implemented as a 2D numpy array where each 1D array represents a single candidate solutions.

One other representation considered during the group phase was such where the value at each index n represents the node to which the n th node points. E.g. $[4, 2, 0, 1, 3]$ represents a cycle $0 \rightarrow 4 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 0$. The problem with this representation is the possibility of having multiple cycles that do not connect together, which complicates the computation of the variation operators.

3.3 Initialization

The population is initialized using a combination of random initialization of candidate solutions, local search, and a form of advanced initialization mechanism. Most of the population (90%) consists of randomly initialized candidate solutions, where a random order of nodes is selected for each candidate solution. Local search is then applied to those candidate solutions and the individuals that are the result of local search become members of the population. The rest of the population (10%) was initialized using Depth First Search (DFS). Combinations of connected nodes were computed in the form of a dictionary-like structure where each node's value is a list of all other nodes it can reach directly. DFS was then carried out using this structure to try and find a legitimate path that does not return an infinite value. This mechanism has two important starting parameters: *startNode* and *previousNodes*. These parameters have to be carefully selected so we do not end up with very similar or even completely equal candidate solutions every time. The parameter *previousNodes* is the list of nodes of candidate solution that have already formed a part of the cycle, and is always just a list with the node 0 in it. Since DFS has to be conducted for each candidate solution we want to initialize, the parameter *startNode* is always a different starting node from the starting nodes used in the previous initializations of candidate solutions. If the starting node would always be the same, we would always initialize the same candidate solution. Another problem is the fact that the dictionary-like structure containing the node combinations is static, and DFS is iterating through this static structure. That means that, even if we choose a different starting node, the algorithm might give very similar solutions. Therefore, all the lists of following nodes were permuted with each iteration of DFS, which removed the possibility that the first node in the list of following nodes is almost always selected and increased population diversity.

This approach should not lead to the candidate solutions initialized by DFS taking over the population immediately because only a small percent of the population is initialized this way, while the vast majority was initialized using only random initialization, which forces the evolutionary algorithm to explore the search space more and maintain sufficient diversity.

One other initialization scheme considered was simply initializing the whole population randomly. However, this could lead to the population not having a single candidate solution whose value is finite when the benchmark problems are large.

The population size was set to 100 candidate solutions, through numerous experiments. This population size works for both the smaller and the larger dataset because it allows for faster execution of each iteration of the evolutionary algorithm, which seems to give better results than if we had a large population size that allowed for a small number of iterations.

3.4 Selection operators

The implemented selection operator is the k-tournament selection. The choice was based on the information provided in the lectures. K-tournament selection will introduce some randomness because of the random selection of a small number of individuals that do not necessarily need to be the best individuals. However, it does choose the best candidate solution from the subset, which increases the probability of producing children with good fitness values. The parameter that needs to be chosen is k , the size of a randomly chosen subset. The value of this parameter was set to 10% of the total population size through several experiments on all datasets.

3.5 Mutation operators

There are three implemented mutation operators: *scramble*, *inverse*, and *swap*. The *scramble* and *inverse* mutations choose a random index from which the shuffle or inverse starts, and a random length of the shuffle/inverse operation that is between 20% and 100% of the whole candidate solution. In the context of the evolutionary algorithm, the three mutation operators alternate between each other through iterations of the algorithm in such a way that every 50 iterations the mutation operator chooses a different function for the mutation. E.g. the first 50 iterations *shuffle* is applied, then the next 50 iterations *reverse* is applied, the following 50 iterations *swap*, and then the next 50 iterations *shuffle* again, etc. This kind of switching introduces randomness because, after a

number of iterations, if the population does not change much, a single mutation function will keep making the same mutations and creating the same candidate solutions as it did before. Introducing different functions for the mutation operator prevents that to some extent.

An important parameter to be selected is the probability of mutation, which was in this case set to 0.4.

3.6 Recombination operators

The recombination operator implemented is the Order Crossover (OX) operator. This operator was chosen based on the article by Abdoun et al. called *Analyzing the Performance of Mutation Operators to Solve the Travelling Salesman Problem*. This operator works as follows: Two parent candidate solutions are selected from the population using the aforementioned selection operator, and two crossover points are randomly chosen along the chromosome. The subsequence between the two crossover points in one parent is copied to the offspring. The remaining positions in the offspring are filled by inheriting the elements from the other parent in the order in which they appear, without violating the order of elements inherited from the first parent. Lastly, to avoid redundancy, any duplicate elements inherited from the first parent are skipped when copying the elements from the second parent. The OX operator preserves sequences from both parents, which could be beneficial if both parents have good partial solutions. On the other hand, it also creates offspring that is different from the parents since new candidate solutions connect nodes that have not been connected in their parents. If there is little overlap between parents, the crossover will still work correctly. The resulting children will be a combination of the distinct parts of the two parents. This recombination has two hyperparameters: the first point of the crossover (randomly chosen), and the length of the crossover, set to 30% of the size of candidate solutions.

One other recombination operator is the Partially-Mapped Crossover (PMX) operator, which mostly works in the same way as the OX operator, with the only difference in avoiding redundancy. PMX handles it in such a way where for each duplicate value in the offspring, the corresponding element from the first parent's subsequence is used to replace it. This operator was discarded because it gave inferior results and the candidate solutions would quickly get stuck in a local optimum.

3.7 Elimination operators

The implemented elimination operator is the $(\lambda + \mu) - \text{elimination}$. This method supports a balance between exploration and exploitation due to the combination of parent and offspring individuals. The selection of candidate solutions that go into the next iteration of the evolutionary algorithm is based on their fitness scores. If the iteration of the evolutionary algorithm is greater than 200, fitness is calculated using fitness sharing.

The parameters that need to be selected are λ and μ . The population size λ is set to be equal to the size of the initial population, and μ is set to be twice the size of λ .

3.8 Local search operators

There are two implemented local search operators. The first local search operator moves the first node that is not 0 through the representation of the candidate solution, until it reaches the end. Each time it swaps places with its following node, the candidate solution's fitness is computed and compared to the previous best candidate solution. E.g. an iteration of this operator would go as follows: $[0, 1, 2, 3] \rightarrow [0, 2, 1, 3] \rightarrow [0, 2, 3, 1]$. This local search operator is used during initialization to try and find a better candidate solution than the randomly initialized one. The other search operator introduces more randomness. It chooses a random index and then iterates through the representation of a candidate solution and swaps each node with the node at the randomly chosen index. E.g. if the randomly chosen index is 3, an iteration of this operator would go as follows: $[0, 1, 2, 3, 4] \rightarrow [0, 3, 2, 1, 4] \rightarrow [0, 3, 1, 2, 4] \rightarrow [0, 3, 1, 4, 2]$. This local search is used in the main loop of the evolutionary algorithm, after applying the mutation operator, and before applying the elimination operator, and only on the candidate solutions that belong to the population of the previous iteration of the evolutionary algorithm. In other words, it is not applied to the offspring.

These two local search operators caused a significant improvement in the performance of the evolutionary algorithm because they make use of the exploitation of the search space and find better solutions that might be in the proximity of the original candidate solutions. However, they do also cause fast convergence of the algorithm, so diversity promotion is absolutely necessary in combination with local search.

Another local search operator considered was iterating through the representation of candidate solution and swapping every neighbouring elements. This operator was discarded because it is computationally time-consuming and does not lead to better results.

3.9 Diversity promotion mechanisms

A diversity promotion scheme implemented is fitness sharing. It works in such a way that the fitness score of each candidate solution is modified based on how many other candidate solutions are near. It was implemented in the elimination operator. A candidate solution was penalized based on how many other candidate solutions that already progressed into the next iteration are near it, using the formula given in the lecture material. This mechanism improves the performance of the evolutionary algorithm because it does not let the population converge too quickly, since it does not allow for very similar candidate solutions to progress into the next iteration. It also prevents the whole population to converge and contain a single candidate solutions.

There are two hyperparameters that need to be determined: σ (radius in which the candidate solutions are considered similar), and α (how strong the fitness penalty is). These parameters were set to 5% of the squared size of candidate solutions, and 0.4, respectively.

The distance of two candidate solutions is calculated as the absolute difference between their unmodified fitness values.

3.10 Stopping criterion

There are two stopping criteria implemented. The first one stops the evolutionary algorithm when the best objective value has not changed in the past 1000 iterations. The second one stops the evolutionary algorithm after a time limit of 5 minutes. For smaller datasets, this ensures that the algorithm does not run for too long if there are no more significant changes. For larger dataset, the problem is not quick convergence, but the fact that the evolutionary algorithm does not converge fast enough, so it will not converge, or even sometimes reach 1000 iterations, and therefore will always be stopped by the time limit.

3.11 Parameter selection

All the parameters mentioned were chosen experimentally, through numerous runs of the evolutionary algorithm, on all datasets.

3.12 Other considerations

The evolutionary algorithm was implemented using Numba, an open-source JIT compiler that translates a subset of Python and numpy into fast machine code. Therefore, the code is very simple and contains a lot of loops and iterations.

4 Numerical experiments

4.1 Metadata

The hyperparameters that needed to be chosen for the evolutionary algorithm are: population size (100), number of offspring (200), σ (5% of the squared size of candidate solutions) and α (0.4) in the fitness sharing scheme, percent of the population initialized by DFS (10%), k in k-tournament (10% of the size of population), length of candidate solution subsequence passed onto the offspring (30% of the size of candidate solutions), and the probability of mutation (40%).

The main characteristic of the computer system on which the evolutionary algorithm was run are as follows: AMD Ryzen 7 6800H with Radeon Graphics - 8 cores, 3.20 GHz, 16.0 GB of RAM (15.2 GB usable) and Python version 3.11.4.

4.2 tour50.csv

After a single run of the evolutionary algorithm, the best tour length found was 26067.37. The corresponding sequence of cities is [0, 32, 31, 2, 45, 24, 18, 9, 5, 8, 46, 35, 16, 4, 15, 47, 42, 11, 3, 19, 38, 10, 13, 7, 41, 14, 1, 43, 30, 48, 6, 22, 49, 20, 29, 37, 17, 40, 34, 33, 44, 12, 27, 36, 23, 28, 25, 39, 21, 26].

The algorithm ran for approximately 42 seconds, reaching about 1500 iterations before stopping, which means that the best objective did not change in the previous 1000 iterations. The convergence graph for this case is represented in figure 1. In terms of time, the evolutionary algorithm converges quite fast for this dataset. In terms of memory, the algorithms is always storing at least 300 integer numpy arrays of size 50. It does not need much more memory that that, except for the initialization, where another 100 numpy arrays are filled with zeros. The initial population is quite diverse, and the mean objective value is high. However, as the iterations progress, it loses its diversity, and the only thing that keeps it is the diversity promotion mechanism. The best acquired solution does beat the greedy heuristic, however, it is not a global optimum since the solution acquired in one of the previous runs had an even better best objective value.

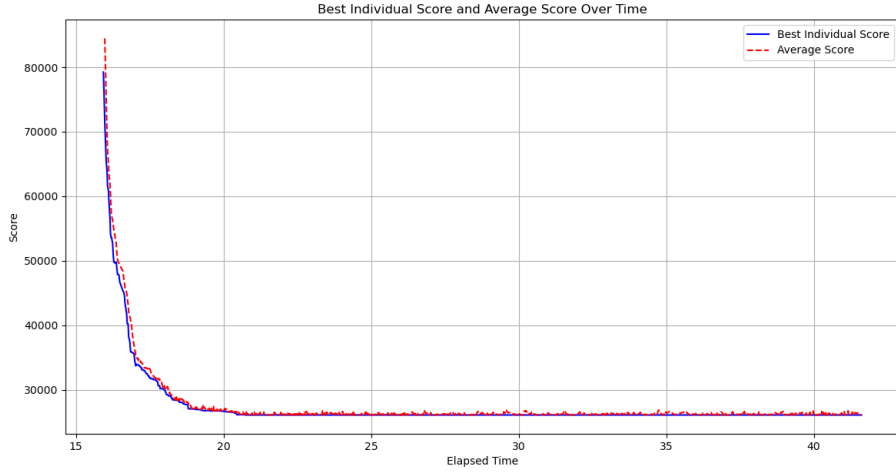


Figure 1: tour50 convergence

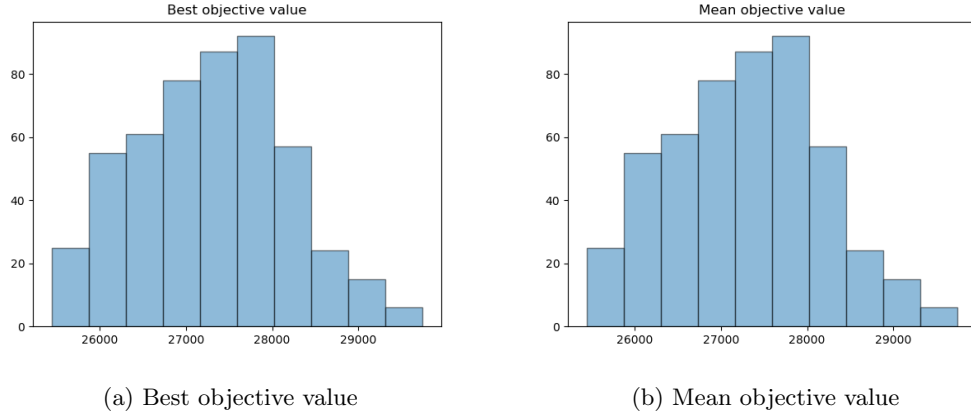


Figure 2: tour50 histograms

Figure 2a represents a histogram of the final best fitnesses, while figure 2b represents a histogram of the final mean fitnesses of 500 runs of the evolutionary algorithm on this benchmark. The mean and standard deviation of the best objective value are 27296.32 and 876.98, while the mean and standard deviation of the mean objective value are 27430.45 and 885.81, respectively. It seems that there is a lot of variability in the results. That might be because a small number of fitness values ends up around 29000. However, most of the final values seem to be roughly between 26000 and 28000. The histograms of the best and the mean objective value are essentially the same, which means that the population converges almost completely at the time of stopping the evolutionary algorithm.

4.3 tour100.csv

After running the evolutionary algorithm on this dataset, the best tour length found is 80750.31, after approximately 35 seconds and 1800 iterations. Figure 3 represents a convergence graph for this run of the algorithm. The algorithm itself converges quite quickly, but so does the population, even though it starts out as a diverse set of candidate solutions. The memory the algorithm needs is just a bit larger than the memory needed for the tour50 problem because the candidate solutions are twice as big. However, the population size and the number of offspring are not changed and no extra memory is needed in that sense. Some operations, like DFS or local search, might take a bit longer to compute, but the time difference is not drastic, since 100 nodes is still pretty manageable. In terms of the final solution itself, it seems like a good fitness score and it beats the greedy heuristic by quite a lot. It might be close to the optimum, since several runs of the evolutionary algorithm on this dataset give similar results.

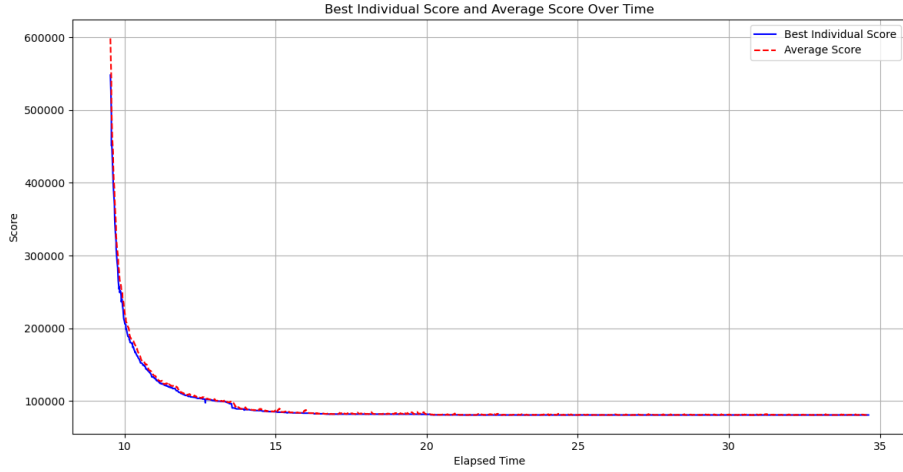


Figure 3: tour100 convergence

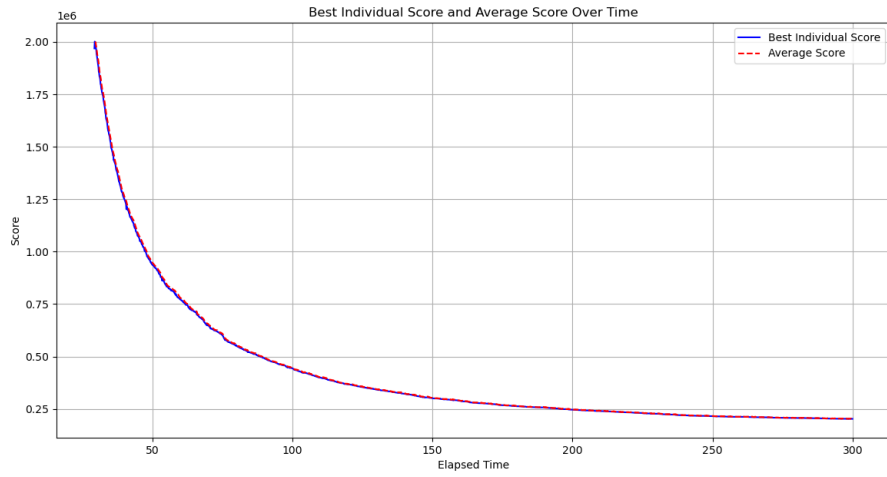


Figure 4: tour500 convergence

4.4 tour500.csv

After running the evolutionary algorithm on the tour500 benchmark, the best tour length found is 203144.35, after approximately 1700 iterations and the time limit of 5 minutes. Figure 4 represents a convergence graph for this run of the algorithm. In contrast to the previously mentioned two benchmarks, the evolutionary algorithm does not converge as fast, it does not even reach full convergence at the time of stopping. It does converge more quickly at the beginning, and then the convergence slows down, but not as significantly as it does with the smaller benchmarks. The memory needed for the execution of the algorithm is again a bit larger just because the candidate solutions are of size 500. However, some operations are now a lot slower because the size of candidate solutions increased, and some of the operations have to iterate through the whole candidate solutions. In terms of the final solution itself, it does not beat the greedy heuristic, but if the algorithm had more time, it might reach it, since it does not converge completely in this case.

4.5 tour1000.csv

After running the evolutionary algorithm on the tour1000 benchmark, the best tour length found is 2108291.88, after approximately 435 iterations and the time limit of 5 minutes. Figure 5 represents a convergence graph for this run of the algorithm. Only the best objective value is visible, because the mean objective value is almost always equal to infinity. The convergence of this graph is very slow in terms of time needed for the execution, but not that slow if we only take into account the number of iterations. The problem with this dataset is that there are a lot of nodes that are not connected, so the initialization and variation operators tend to create paths that are not valid. DFS creates legitimate paths, however, for the exact same reason, it is very computationally

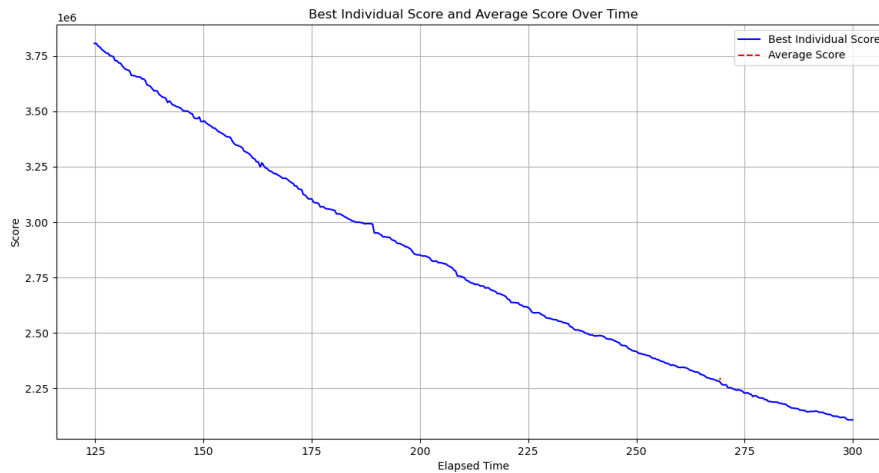


Figure 5: tour1000 convergence

time-consuming. The memory needed for execution of the algorithm is again not extensive, since none of the parameters, except for the size of candidate solutions have not changed. In terms of the final solution, it does not beat the heuristic since the time frame is not big enough for this evolutionary algorithm to find optimal solutions for such large datasets.

5 Critical reflection

Three main strengths of evolutionary algorithms:

1. Balance between exploration and exploitation of the search space
2. Ability to perform well on different kinds of problems
3. No need for heavy mathematical computations

Three main weak points of evolutionary algorithms:

1. Sensitivity to parameter choices
2. Computational complexity, especially for large datasets
3. Quick convergence to local optima

This project taught me that evolutionary algorithms can be made to find solutions to complex problems without having to compute heavy mathematical equations (like gradient descent). It also taught me that they are extremely sensitive to hyperparameter choices and that it is very important and difficult to find the right combination that would result in a good solution. What surprised me is that even the slightest change in one of the hyperparameters could cause an extreme change in the choice of candidate solutions and the convergence to the final best solution. Also, changing one hyperparameter often leads to having to change others as well. I also found out that, without diversity promotion, the whole population tends to quickly converge to a local optimum.

I believe that evolutionary algorithms could be appropriate for the Travelling Salesman Problem under certain conditions. The hyperparameters have to be carefully selected, and they could even depend on the size of the dataset. However, for smaller datasets, traditional methods might work better, since they are constructed to find a global optimum, unlike evolutionary algorithms. On the other hand, for larger datasets, that could be extremely computationally heavy, which makes evolutionary algorithms a better choice. The best approach to solving the Travelling Salesman problem might actually be a combination of an evolutionary and a traditional algorithm.